Web tests do an excellent job of illustrating how a user interacts with the system via the user interface to achieve a particular business goal. But they don't need to show every possible path through the system—just the more significant ones. More exhaustive testing can be left to faster-running unit tests.

Web tests can also illustrate how business rules are reflected in the user interface. For example, suppose that when they book a flight, Frequent Flyer members should be given the option to choose their seat, a privilege not offered to other customers. This would be a good candidate for an automated web test.

A good rule of thumb is to ask yourself whether you're illustrating how the user interacts with the application or underlying business logic that's independent of the user interface. For example, suppose you were testing a user authentication feature. The acceptance criteria might include the following:

- The user should receive feedback indicating the strength of the password entered.
- Only strong passwords should be accepted.

The first acceptance criterion relates to the user's interaction with the web page, and would need to illustrate how this feedback is provided on the login page. This would be a good candidate for an automated web test.

The second criterion, on the other hand, is about determining what makes a strong password, and what passwords users should be allowed to enter. While this could be done through the user interface by repeatedly submitting different passwords, this would be wasteful. What you're really checking here is the password-strength algorithm, so an application-code-level test would be more appropriate.

8.2 Automating web-based acceptance criteria using Selenium WebDriver

In this section, we'll look at automating web tests using Selenium WebDriver. Selenium WebDriver is a popular open source web browser automation library that can be used to write effective, automated web tests. It also forms the basis for many higher-level web-testing tools. The examples will focus on working with WebDriver in Java, but the principles and techniques we'll discuss will be generally applicable to any WebDriver-based testing.

WebDriver is a browser-automation tool. It lets you write tests that launch and interact with a real browser. This interaction can include simple clicks on buttons or links, or more sophisticated mouse operations such as hovering or dragging and dropping.

WebDriver lets you check the test outcomes by inspecting the state of the page in the browser. WebDriver also gives you the ability to take screenshots along the way—screenshots that can be used later as part of the test reports or living documentation.

Figure 8.4 shows a high-level view of WebDriver. WebDriver supports a large number of web browsers, including Firefox, Chrome, and Internet Explorer. This allows you to test your application in different environments and with different browsers.